

Kotlin

Advanced

Павел Галанин
2026



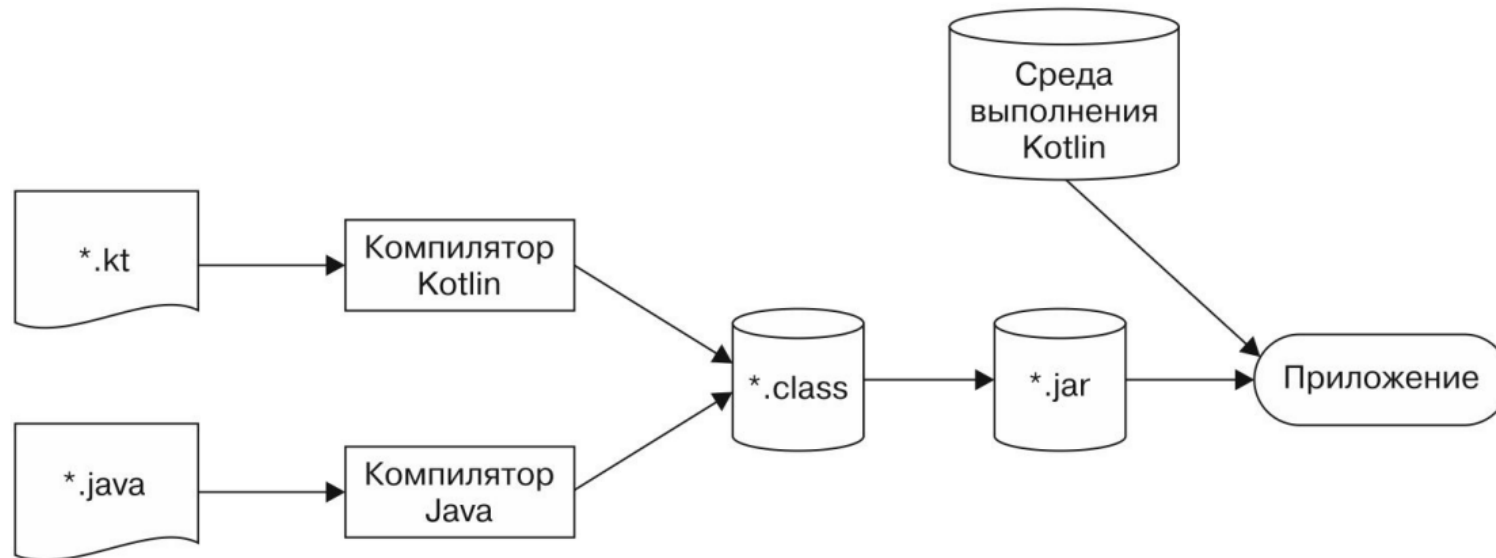
О чём поговорим

- Generics
- Лямба-выражения
- Делегаты
- Рефлексия
- Kotlin misc.

Kotlin и Java

Kotlin и Java – языки, которые идут в паре, их **можно использовать в одном проекте**.

И Kotlin, и Java компилируются в набор `.class` файлов компилятором `kotlin` и `java` соответственно, из которых в дальнейшем собирается `.jar`-файл.



Generics

Generic-типы: зачем?

// ❌ Без дженериков (Опасно)

```
class RegistrationDesk {  
    private val participants = mutableListOf<Any>()  
    fun register(person: Any) { participants.add(person) }  
    fun getFirst(): Any = participants.first()  
}
```

// ✅ С дженериками (Безопасно)

```
class RegistrationDesk<T> {  
    private val participants = mutableListOf<T>()  
    fun register(person: T) { participants.add(person) }  
    fun getFirst(): T = participants.first()  
}
```

Generic-типы: синтаксис

```
class RegistrationDesk<T> {  
    private val participants = mutableListOf<T>()  
  
    fun register(participant: T) {  
        participants.add(participant)  
    }  
}
```

```
fun <E> printStartingOrder(elements: List<E>) {  
    elements.forEachIndexed { index, element ->  
        println("${index + 1} - $element")  
    }  
}
```

```
val <T> List<T>.firstParticipant: T?  
    get() = this.firstOrNull()
```

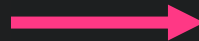
- **Стандартные обозначения:**

- **T (Type)** — базовый универсальный тип
- **E (Element)** — элемент коллекции (List<E>, Set<E>)
- **K (Key)** — ключ (Map<K, V>)
- **V (Value)** — значение в словаре.
- **R (Result)** — тип результата (часто - возвращаемое значение)

Generic-типы: стирание типов

```
// Наш красивый, типизированный Kotlin-код
fun processAthletes() {
    val dancers: List<RockAndRollDancer> = listOf(
        RockAndRollDancer("Анна", "Спартак")
    )

    // Компилятор знает тип, нам не нужно делать cast
    val firstDancer: RockAndRollDancer = dancers[0]
    println(firstDancer.club)
}
```



```
// Суровая реальность после компиляции (Type Erasure в действии)
public static final void processAthletes() {
    // 1. Тип List<RockAndRollDancer> стерся до просто List
    List dancers = CollectionsKt.listOf(
        new RockAndRollDancer("Анна", "Спартак")
    );

    // 2. Метод get() возвращает Object (в Kotlin - Any)
    // 3. Компилятор молча вставил cast: (RockAndRollDancer)
    RockAndRollDancer firstDancer = (RockAndRollDancer) dancers.get(0);

    System.out.println(firstDancer.getClub());
}
```

Generic-типы

- ✓ позволяют использовать единую логику для разных типов объектов с сохранением безопасности типов
- ✓ могут определяться для всего класса, для конкретной функции или поля
- ✓ **стирание типов** (type erasure) – информация о generic-типе не хранится в скомпилированном коде: данные хранятся в виде Any (Object), в местах использования – приведение к нужному типу

```
class Cage<out T : Animal>(val animal: T)
```

```
public inline fun <T> Iterable<T>.filter(predicate: (T) → Boolean): List<T> {  
    return filterTo(ArrayList<T>(), predicate)  
}
```


Ограничения типов

```
// Ограничиваем T: теперь T обязан быть наследником Athlete
fun <T : Athlete> countAthletesByClub(athletes: List<T>, targetClub: String): Int {
    // Благодаря <T : Athlete>, мы можем безопасно обращаться к свойству it.club
    return athletes.count { it.club == targetClub }
}

val dancers = listOf(
    Dancer( name: "Иван", club: "Спартак"),
    Footballer( name: "Олег", club: "Динамо")
)

val spartakCount = countAthletesByClub(dancers, targetClub: "Спартак") // Работает!
// countAthletesByClub(listOf(1, 2, 3), "Спартак") // ❌ Ошибка компиляции
```

```
public inline fun <T> Iterable<T>.filter(predicate: (T) → Boolean): List<T> {
    return filterTo(ArrayList<T>(), predicate)
}
```

Проблема инвариантности

Если тип B является подтипом A ,
является ли тип $\text{Generic}\langle B \rangle$ подтипом $\text{Generic}\langle A \rangle$?

Инвариантность

B подтип $A \rightarrow \text{Generic}\langle B \rangle$ **не подтип** $\text{Generic}\langle A \rangle$

Ковариантность

B подтип $A \rightarrow \text{Generic}\langle B \rangle$ **подтип** $\text{Generic}\langle A \rangle$

Контрвариантность

B подтип $A \rightarrow \text{Generic}\langle A \rangle$ **подтип** $\text{Generic}\langle B \rangle$

Инвариантность

По умолчанию дженерики инварианты: нельзя положить `List<Int>` в переменную `List<Number>`, хотя `Int` наследуется от `Number`.

Но почему нельзя?

```
val dancers: MutableList<Dancer> = mutableListOf()
```

```
// ❌ Ошибка компиляции! Инвариантность.
```

```
// val athletes: MutableList<Athlete> = dancers
```

```
// Если бы это было разрешено, мы бы сломали типизацию:
```

```
// athletes.add(Footballer("Игорь", "Зенит"))
```

```
// val dancer: Dancer = dancers[0] // ClassCastException!
```

Ковариантность (out)

Ковариантность: В подтип $A \rightarrow \text{Generic}\langle B \rangle$ подтип $\text{Generic}\langle A \rangle$

$\langle \text{out } T \rangle \rightarrow$ класс может только возвращать (производить) элементы типа T

```
// OUT: Ковариантность (Производитель)
// Только отдает участников, ничего не принимает
interface RegistrationList<out T> {
    fun getNextParticipant(): T

    // ❌ ОШИБКА КОМПИЛЯЦИИ!
    // Type parameter T is declared as 'out' but occurs in 'in' position
    fun addParticipant(item: T)
}
```

out = Producer (только на выдачу)

Контрвариантность (in)

Контрвариантность

В подтип A $\rightarrow$ Generic<A> подтип Generic

<in T> $\rightarrow$ класс может только принимать (потреблять) элементы типа T

```
interface JudgingSheet<in T> {  
    fun submitScore(participant: T, score: Double)  
}
```

in = Consumer (только на приём)

Инвариантность

Может ли быть одновременно in и out?

Сравнение с Java

- **Raw Types (Сырые типы):**

Java: Разрешены (`List list = new ArrayList();`).

Kotlin: **Строго запрещены!** Тип (или проекция) обязателен всегда.

- **Variance (Вариативность):**

Java: *Use-site variance* (Wildcards: `? extends T`, `? super T`) — пишем при использовании.

Kotlin: *Declaration-site variance* (`out`, `in`) — пишем один раз при объявлении класса.

- **Неизвестный тип:**

Java: Неограниченный wildcard `<?>`.

Kotlin: Звездная проекция `<*>`.

- **Доступ к типу в Runtime (Type Erasure):**

Java: Тип всегда стирается. Нужно передавать `Class<T>`.

Kotlin: Тип стирается, НО можно сохранить его через `inline + reified`.

- **Множественные ограничения (Upper Bounds):**

Java: `<T extends Class & Interface>`

Kotlin: Изящное ключевое слово `where`.

Лямбда-выражения

Функции высшего порядка

- ✓ Функция высшего порядка - это функция, которая принимает функции как параметры, и/или возвращает функцию в качестве результата.

```
public inline fun <T> Iterable<T>.filter(predicate: (T) → Boolean): List<T> {  
    return filterTo(ArrayList<T>(), predicate)  
}
```

inline функции



Функции помеченный ключевым словом **inline** на этапе компиляции встраивается в место вызова. Это позволяет экономить ресурсы если в функции есть лямбды, так как не тратится память на их создание



reified - можно смотреть тип дженериков, так как код встраивается в место вызова и мы точно знаем тип

```
inline fun measureExecutionTime(block: () -> Unit) {  
    val start = System.currentTimeMillis()  
    block() // Код лямбды встроится прямо сюда  
    println("Выполнение заняло ${System.currentTimeMillis() - start} мс")  
}
```

```
inline fun <reified T : Activity> Context.startActivity() {  
    val intent = Intent(this, T::class.java)  
    startActivity(intent)  
}
```

```
// Использование: элегантно и без лишних параметров  
context.startActivity<DetailsActivity>()
```

inline функции: noinline, crossinline

✍ **noinline** - при нескольких аргументах с лямбда помечаем те, которые не должны встраиваться

✍ **crossinline** - запрет использования return в лямбде:
так как код встраивается в место вызова, return из лямбды просто завершит функцию

```
inline fun processRoutine(  
    performance: () -> Unit,           // Эта лямбда встроится (inline)  
    noinline onFinish: () -> Unit    // Эту мы хотим сохранить как объект  
) {  
    println("Музыка пошла!")  
    performance() // Код скопируется сюда  
  
    // ❌ Без noinline это вызвало бы ошибку:  
    // "Illegal usage of inline-parameter"  
    savedCallback = onFinish  
}
```

Делегаты

Делегаты свойств

Делегаты свойств позволяют вынести логику кастомных геттеров/сеттеров в отдельный класс

```
// Классический подход (Много бойлерплейта)
private var _clubName: String? = null
var clubName: String
    get() {
        if (_clubName == null) _clubName = loadClubFromDb()
        return _clubName!!
    }
    set(value) {
        _clubName = value
        saveClubToDb(value)
    }
```

```
// Подход Kotlin (Изящно)
// Вся логика скрыта внутри DatabaseDelegate
var clubName: String by DatabaseDelegate("club_key")
```

```
class DatabaseDelegate(private val key: String) {

    // Это тот самый backing field (_clubName из первого примера),
    // но теперь он инкапсулирован внутри делегата.
    private var _value: String? = null

    operator fun getValue(thisRef: Any?, property: KProperty<*>): String {
        if (_value == null) {
            _value = loadClubFromDb(key)
        }
        return _value!!
    }

    operator fun setValue(thisRef: Any?, property: KProperty<*>, value: String) {
        _value = value
        saveClubToDb(key, value)
    }
}
```

Делегат lazy

lazy откладывает инициализацию тяжелого объекта до момента первого обращения к нему, а затем кэширует результат

```
class JudgingManager {  
    // Тяжелая операция: загружаем и парсим огромную таблицу  
    // судейских назначений только тогда, когда она реально понадобится  
    val assignmentsMasterFile by lazy {  
        SpreadsheetParser.load("assignments_v2.xlsx")  
    }  
  
    // Оптимизация для Android Main Thread  
    val titleView: TextView by lazy(LazyThreadSafetyMode.NONE) {  
        findViewById(R.id.title_text)  
    }  
}
```

Примеры делегатов

observable — позволяет добавить слушатель на изменение переменной

vetoable — если код вернёт false, на новое значение будет наложено "вето", и переменная сохранит старое значение.

```
class PromoController {  
    // observable: Реагируем на изменение флага  
    var isPromoVisible: Boolean by Delegates.observable(false) { _, old, new ->  
        if (new && !old) {  
            showHalfScreenPromoSheet() // Показываем шторку  
        } else if (!new && old) {  
            hideHalfScreenPromoSheet() // Прячем шторку  
        }  
    }  
  
    // vetoable: Защита от некорректных данных  
    // Оценка за акробатику не может быть отрицательной или больше 50  
    var acrobaticsScore: Int by Delegates.vetoable(0) { _, _, new ->  
        new in 0..50  
    }  
}
```

Делегаты в Android

```
class ProfileFragment : Fragment() {  
  
    // KTX Delegate: сам найдет ViewModelProvider, проверит скоуп  
    // Фрагмента/Активности и отдаст нужный инстанс  
    private val viewModel: ProfileViewModel by viewModels()  
  
    // KTX Delegate: лениво достанет аргументы из Bundle  
    private val args: ProfileFragmentArgs by navArgs()  
  
    override fun onCreateView(view: View, savedInstanceState: Bundle?) {  
        super.onCreateView(view, savedInstanceState)  
        // Используем делегаты как обычные переменные  
        viewModel.loadProfile(args.userId)  
    }  
}
```


Рефлексия

Рефлексия

Рефлексия (Reflection) — это механизм, который позволяет программе изучать саму себя во время выполнения (runtime):

- Узнать методы объекта: получить полный список функций или найти конкретную по её строковому имени
- Узнать свойства (поля) класса: прочитать список всех свойств, которые есть в программе у данного класса
- Читать и изменять значения: динамически получить значение свойства (get) или записать в него новое, даже не зная точного типа объекта на этапе компиляции
- Вызывать функции "вслепую": запустить выполнение найденного метода, передав ему нужные аргументы во время работы программы
- Обходить инкапсуляцию (Взлом private): находить приватные поля или методы, принудительно делать их доступными (через `isAccessible = true`) и взаимодействовать с ними (часто используется при написании хитрых Unit-тестов)
- Создавать объекты динамически: инстанцировать классы, получая доступ к их конструкторам во время выполнения программы, не вызывая конструктор напрямую в коде

Reflection API

Есть API для рефлексии в самой Java, но Kotlin предоставляет библиотеку `kotlin-reflect`, дополняющую специфичными для Kotlin возможностями (информация о дата-классах, nullable-типах и т.д.).

Нюансы и проблемы рефлексии

- Потеря типобезопасности: ошибки выявляются только в рантайме
- Снижение производительности: динамический поиск методов обходится дорого
- Проблемы с обфускацией (ProGuard / R8): минификация ломает поиск по строковым именам
- Хрупкость архитектуры: зависимость от внутренних (приватных) реализаций чужого кода
- Размер APK: библиотека kotlin-reflect заметно увеличивает вес приложения

Kotlin misc.

Scope функции

Scope-функции — это способ выполнить блок кода в контексте конкретного объекта.

- ✍ **let** - часто используется для безопасного выполнения блока кода с null-выражениями
- ✍ **also** - используется для выполнения каких-либо дополнительных действий
- ✍ **apply** - настроить объект и не надо возвращать результат
- ✍ **run** - настроить объект и вычислить результат
- ✍ **with** - используется для объединения вызовов функций объекта

	Функция будет принимать this	Функция будет принимать it
Будет возвращен объект на котором вызывается функция (self)	apply	also
Будет возвращен результат функции (result)	run, with	let

infix-функции

Делают код выглядящим как обычный текст на английском языке:

```
// Объявляем infix-функцию расширения для объединения танцоров в пару
infix fun Dancer.partnersWith(partner: Dancer): Pair<Dancer, Dancer> {
    return Pair(this, partner)
}

fun main() {
    val ivan = Dancer("Иван", "Спартак")
    val anna = Dancer("Анна", "Спартак")

    // Классический вызов:
    // val couple = ivan.partnersWith(anna)

    // Изящный infix-вызов (читается как предложение):
    val couple = ivan partnersWith anna

    println("Паpa: ${couple.first.name} и ${couple.second.name}")
}
```

Operator overloading

Позволяет использовать стандартные математические операторы (+, -, >, []) для произвольных объектов

```
// Класс оценки выступления
data class RoutineScore(val acrobatics: Int, val dance: Int) {

    // Перегружаем оператор сложения (+)
    operator fun plus(other: RoutineScore): RoutineScore {
        return RoutineScore(
            acrobatics = this.acrobatics + other.acrobatics,
            dance = this.dance + other.dance
        )
    }
}

fun main() {
    val technicalJudgeScore = RoutineScore(acrobatics = 25, dance = 10)
    val artisticJudgeScore = RoutineScore(acrobatics = 5, dance = 30)

    // Выглядит как встроенный тип! Вызывается функция plus()
    val finalScore = technicalJudgeScore + artisticJudgeScore

    println("Итог: Акробатика=${finalScore.acrobatics}, Танец=${finalScore.dance}")
}
```


Value class

Оптимизированный способ создавать классы-обёртки для набора значений:
«выглядит» как обычный класс с полями, но при компиляции «разворачивается» в типы значений.

```
@JvmInline  
value class JudgeId(val id: String)
```

```
@JvmInline  
value class AthleteId(val id: String)
```

```
@JvmInline  
value class Score(val points: Int) {  
    init { require(points >= 0) { "Оценка не может быть отрицательной" } }  
}
```

Type alias

Заменяет «страшные» названия типов данных на лаконичные:

```
// ❌ Без псевдонимов (Тяжело читать и легко ошибиться)
fun processResults(results: Map<String, List<Pair<Athlete, Int>>>) { ... }

// ✅ Создаем смысловые псевдонимы
typealias CategoryName = String
typealias TotalScore = Int
typealias JudgingProtocol = Map<CategoryName, List<Pair<Athlete, TotalScore>>>

// Теперь контракт функции понятен с первого взгляда:
fun processResults(results: JudgingProtocol) {
    // Логика обработки
}
```

Спасибо
за внимание!

